

# Python Assignment-21

1: Design a Python application that creates two threads named Prime and NonPrime.

- Both threads should accept a list of integers.
- The Prime thread should display all prime numbers from the list.
- The NonPrime thread should display all non-prime numbers from the list.

Answer :

```
#
=====
===
# Author   : codingwithamar@gmail.com
# File      : A21Q1.py
# Path       : Marvellous-Python-Assignments-P229\Assignment-21\A21Q1.py
# Subject/Question : 1: Design a Python application that creates two
threads named Prime and NonPrime.
# Description : • Both threads should accept a list of integers.
# • The Prime thread should display all prime numbers from the list.
# • The NonPrime thread should display all non-prime numbers from the
list.
#
=====
===

import threading
import queue

def Prime(iValues,iQT1):
    PrimeList = []
    for value in iValues:
        count = 0
        for i in range(1,value+1):
            if value%i==0:
                count = count + 1
        if count == 2:
            PrimeList.append(value)
    iQT1.put(PrimeList)

def NonPrime(iValues,iQT2):
    NonPrimeList = []
    for value in iValues:
        count = 0
        for i in range(1,value+1):
            if value%i==0:
                count = count + 1
        if count != 2:
```

```

        NonPrimeList.append(value)
    iQT2.put(NonPrimeList)

def main():
    print("Enter the list of numbers : ")
    Values = list(map(int,input().split()))
    print("Entered list is : ", Values)

    QT1 = queue.Queue()
    QT2 = queue.Queue()

    T1 = threading.Thread(target = Prime, name = "Prime", args =
(Values,QT1))
    T2 = threading.Thread(target = NonPrime, name = "NonPrime", args =
(Values,QT2))

    T1.start()
    T2.start()
    T1.join()
    T2.join()

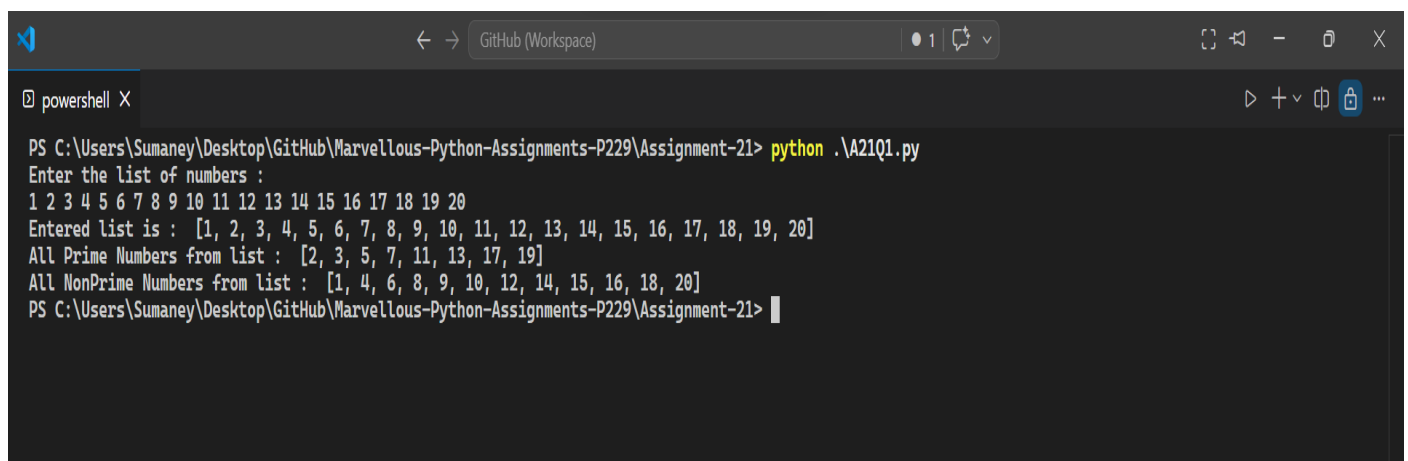
    RPrime = QT1.get()
    RNonPrime = QT2.get()

    print("All Prime Numbers from list : ",RPrime)
    print("All NonPrime Numbers from list : ", RNonPrime)

if __name__ == "__main__":
    main()

```

OUTPUT :



```

PS C:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21> python .\A21Q1.py
Enter the list of numbers :
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Entered list is : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
All Prime Numbers from list : [2, 3, 5, 7, 11, 13, 17, 19]
All NonPrime Numbers from list : [1, 4, 6, 8, 9, 10, 12, 14, 15, 16, 18, 20]
PS C:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21>

```

## 2: Design a Python application that creates two threads.

- Thread 1 should calculate and display the maximum element from an list.
- Thread 2 should calculate and display the minimum element from the same list.
- The list should be accepted from the user.

Answer :

```
#
=====
===
# Author   : codingwithamar@gmail.com
# File      : A21Q2.py
# Path      : Marvellous-Python-Assignments-P229\Assignment-21\A21Q2.py
# Subject/Question : 2: Design a Python application that creates two
threads.
# Description :
# • Thread 1 should calculate and display the maximum element from an
list.
# • Thread 2 should calculate and display the minimum element from the
same list.
# • The list should be accepted from the user.
#
=====
===

import threading
import queue

def Max(iValues,iTQ1):
    MaxNumber = max(iValues)
    iTQ1.put(MaxNumber)

def Min(iValues,iTQ2):
    MinNumber = min(iValues)
    iTQ2.put(MinNumber)

def main():
    print("Enter the list of Numbers")
    Values = list(map(int,input().split()))
    print("You entered list is  :",Values)

    TQ1 = queue.Queue()
    TQ2 = queue.Queue()

    T1 = threading.Thread(target=Max, name="Maximum", args = (Values,TQ1))
    T2 = threading.Thread(target=Min, name="Minimum", args = (Values,TQ2))

    T1.start()
```

```

T2.start()
T1.join()
T2.join()

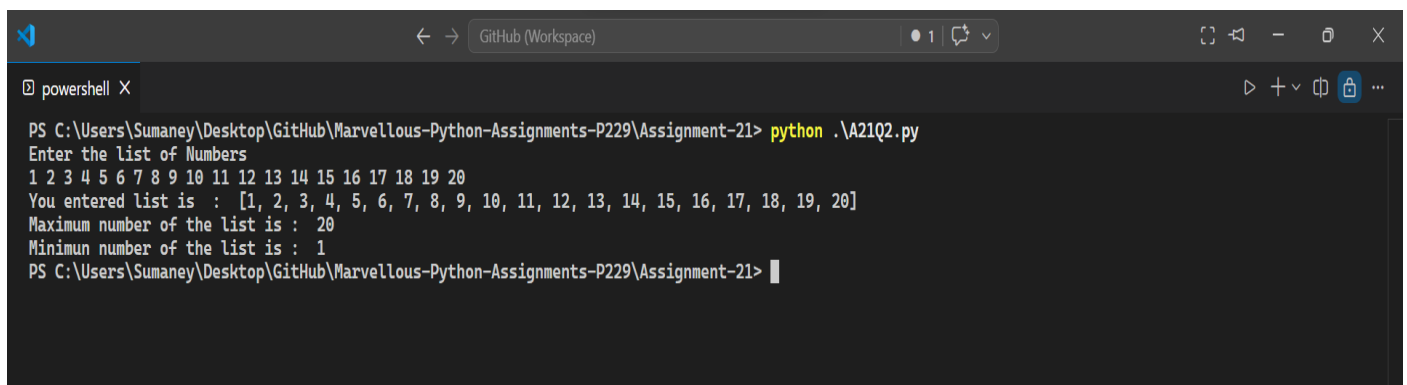
MaxNumber = TQ1.get()
MinNumber = TQ2.get()

print("Maximum number of the list is : ",MaxNumber)
print("Minimun number of the list is : ",MinNumber)

if __name__ == "__main__":
    main()

```

OUTPUT :



```

PS C:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21> python .\A21Q2.py
Enter the List of Numbers
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
You entered list is : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
Maximum number of the list is : 20
Minimun number of the list is : 1
PS C:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21>

```

**3: Design a Python application where multiple threads update a shared variable.**

- Use a **Lock** to avoid race conditions.
- Each thread should increment the shared counter multiple times.
- Display the final value of the counter after all threads complete execution.

Answer :

```

import threading

Counter = 0
Lock = threading.Lock()

def Increment():
    global Counter
    for i in range(100000):
        Lock.acquire()
        Counter += 1
        Lock.release()

def main():
    global Counter

```

```

T1 = threading.Thread(target=Increment)
T2 = threading.Thread(target=Increment)
T3 = threading.Thread(target=Increment)

T1.start()
T2.start()
T3.start()

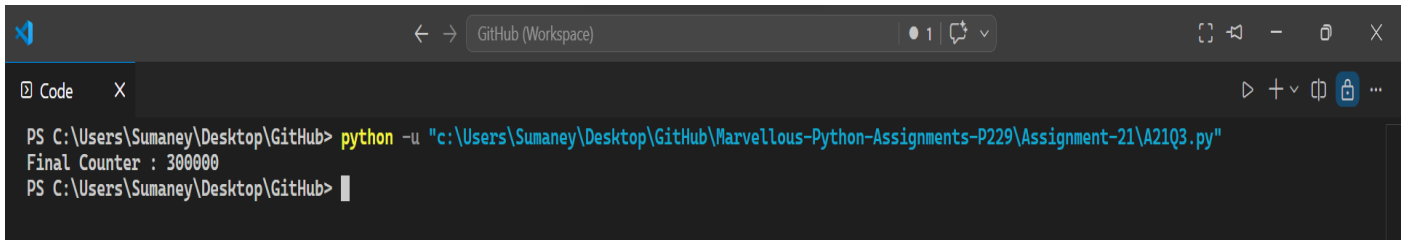
T1.join()
T2.join()
T3.join()

print("Final Counter :", Counter)

if __name__ == "__main__":
    main()

```

OUTPUT :



```

PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21\A21Q3.py"
Final Counter : 3000000
PS C:\Users\Sumaney\Desktop\GitHub>

```

#### 4. Design a Python application that creates two threads.

- Thread 1 should compute the sum of elements from a list.
- Thread 2 should compute the product of elements from the same list.
- Return the results to the main thread and display them.

Answer :

```

import threading
# -----
---
# Thread Class for Sum
# -----
---
class SumThread(threading.Thread):

    def __init__(self, Data):
        super().__init__()
        self.Data = Data
        self.Result = 0

```

```

def run(self):

    Total = 0

    for Value in self.Data:
        Total += Value

    self.Result = Total

# -----
---
# Thread Class for Product
# -----
---
class ProductThread(threading.Thread):

    def __init__(self, Data):
        super().__init__()
        self.Data = Data
        self.Result = 1

    def run(self):

        Product = 1

        for Value in self.Data:
            Product *= Value

        self.Result = Product

# -----
---
# Main Function
# -----
---
def main():

    Data = [1, 2, 3, 4, 5]

    print("Input List :", Data)

    T1 = SumThread(Data)
    T2 = ProductThread(Data)

```

```
T1.start()
T2.start()

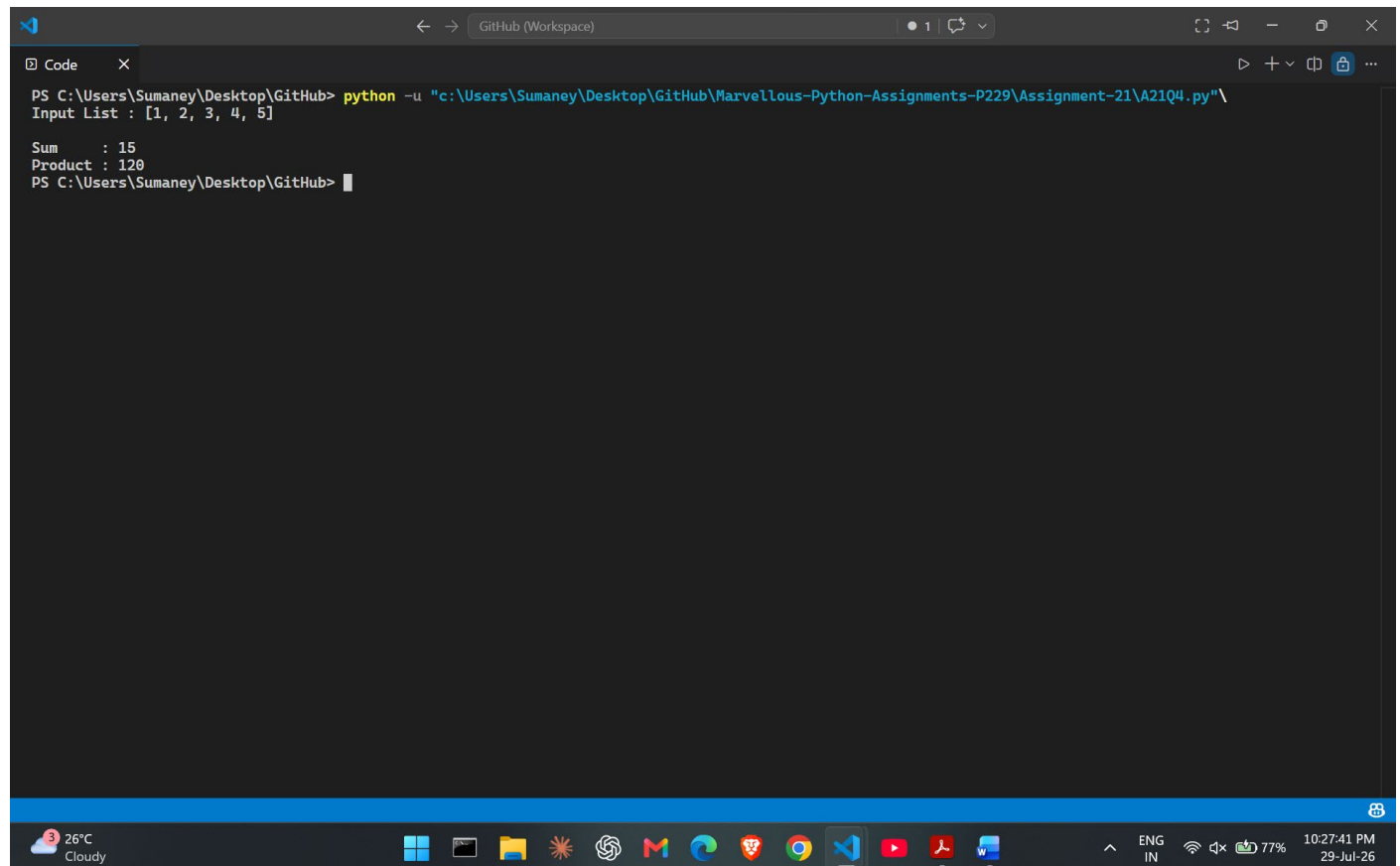
T1.join()
T2.join()

print("\nSum      :", T1.Result)
print("Product :", T2.Result)

# -----
-----

if __name__ == "__main__":
    main()
```

OUTPUT :



```
PS C:\Users\Sumaney\Desktop\GitHub> python -u "c:\Users\Sumaney\Desktop\GitHub\Marvellous-Python-Assignments-P229\Assignment-21\A21Q4.py"
Input List : [1, 2, 3, 4, 5]

Sum      : 15
Product : 120
PS C:\Users\Sumaney\Desktop\GitHub>
```